

Digital Image Processing - Lab Report 2: Depth Extraction from Stereoscopic Images

Papachristodoulou Georgia, AEM: 11177

PART 1: Helper function that it was introduced

To optimize the codebase, adhere to the DRY (Don't Repeat Yourself) principle, and simplify the local descriptor extraction, a helper function named `_get_circle_values` was introduced. This function encapsulates the mathematical logic required to sample N points along a circle of a given radius ρ centered at a specific pixel. Crucially, it implements the bilinear interpolation formula to accurately estimate the sub-pixel intensity values when the circle intersects the image grid. Abstracting this logic significantly streamlined the implementation of both the basic and upgraded local descriptors.

Python Implementation of `_get_circle_values`:

```
def _get_circle_values(I: np.ndarray, px: float, py: float, width: int, height: int, rho: float, N: int, angle_step: float) -> np.ndarray:
    """
    Helper function: Extracts N interpolated values along a circle
    with radius 'rho' centered at (px, py).
    """
    circle_values = np.zeros(N)
    for i in range(N):
        # Calculate coordinates for each point on the circle
        theta = i * angle_step
        cx = px + rho * math.cos(theta)
        cy = py + rho * math.sin(theta)

        # Find the 4 nearest neighbors for bilinear interpolation
        x0 = int(math.floor(cx))
        y0 = int(math.floor(cy))

        x1 = min(x0 + 1, width - 1)
        y1 = min(y0 + 1, height - 1)

        dx = cx - x0
        dy = cy - y0

        # Retrieve pixel intensities
        I00 = float(I[y0, x0])
        I10 = float(I[y0, x1])
        I01 = float(I[y1, x0])
        I11 = float(I[y1, x1])

        # Apply bilinear interpolation formula
        circle_values[i] = (I00 * (1 - dx) * (1 - dy) +
                             I10 * dx * (1 - dy) +
                             I01 * (1 - dx) * dy +
                             I11 * dx * dy)

    return circle_values
```

PART 2: Functions

1. Basic Local Descriptor (`myLocalDescriptor`)

The `myLocalDescriptor` function implements a rotation-invariant feature descriptor as requested in Section 1.1 of the assignment. For a given pixel p , the function samples concentric circles with radii ranging from ρ to ρ_M using a step of ρ_s . For each circle, N points are sampled using the aforementioned

helper function, and the mean intensity value is computed to form the final descriptor vector. Boundary checks were implemented to return an empty array if the largest circle exceeds the image dimensions, ensuring robust execution.

Python Implementation of myLocalDescriptor:

```
def myLocalDescriptor(I: np.ndarray, p: tuple, rhom: float, rhoM: float, rhostep: float, N: int) -> np.ndarray:
    """
    Basic local descriptor: Calculates the mean intensity along concentric circles.
    """
    px, py = p
    height, width = I.shape

    # Check if the largest circle exceeds image boundaries
    if (px - rhoM < 0 or px + rhoM >= width or py - rhoM < 0 or py + rhoM >= height):
        return np.array([])

    # Generate radii array and initialize descriptor
    radii = np.arange(rhom, rhoM, rhostep)
    descriptor = np.zeros(len(radii))
    angle_step = 2 * math.pi / N

    # Calculate the mean value for each circle using the helper function
    for idx, rho in enumerate(radii):
        circle_values = _get_circle_values(I, px, py, width, height, rho, N, angle_step)
        descriptor[idx] = np.mean(circle_values)

    return descriptor
```

2. Upgraded Local Descriptor (myLocalDescriptorUpgrade):

Justification for the Upgrade: The basic descriptor relies solely on the mean intensity of each concentric circle, which discards valuable information regarding the local texture and contrast. For instance, a circle crossing a sharp edge might have the same mean intensity as a circle in a uniformly gray area. To enrich the descriptor without excessively increasing computational complexity, the myLocalDescriptorUpgrade function was engineered to extract both the **mean** and the **standard deviation** of the interpolated values along each circle. The standard deviation captures the intensity variance (contrast/texture) along the circumference. The resulting descriptor vector has double the length of the basic version, alternating between mean and standard deviation values for each consecutive radius, thus providing a significantly more robust signature for stereo matching.

Python Implementation of myLocalDescriptorUpgrade:

```
def myLocalDescriptorUpgrade(I: np.ndarray, p: tuple, rhom: float, rhoM: float, rhostep: float, N: int) -> np.ndarray:
    """
    Upgraded local descriptor: Calculates both mean and standard deviation
    along concentric circles for richer texture information.
    """
    px, py = p
    height, width = I.shape

    # Check boundary conditions
    if (px - rhoM < 0 or px + rhoM >= width or py - rhoM < 0 or py + rhoM >= height):
        return np.array([])

    # Initialize descriptor with double the size (mean + std for each circle)
    radii = np.arange(rhom, rhoM, rhostep)
    descriptor = np.zeros(len(radii) * 2)
    angle_step = 2 * math.pi / N

    # Populate the descriptor array using the helper function
    for idx, rho in enumerate(radii):
```

```

circle_values = _get_circle_values(l, px, py, width, height, rho, N, angle_step)

# Store mean at even indices and standard deviation at odd indices
descriptor[2*idx] = np.mean(circle_values)
descriptor[2*idx + 1] = np.std(circle_values)

return descriptor

```

3. Point-wise Corner Verification (isCorner):

The `isCorner` function evaluates whether a specific spatial coordinate $p=(x,y)$ constitutes a corner feature based on the Harris corner detection metric. It extracts a localized 5x5 window centered around the point and computes the partial spatial derivatives I_x and I_y using 3x3 Sobel operators. Subsequently, the elements of the structure tensor $M (S_{xx}, S_{yy}, S_{xy})$ are accumulated. The final corner response R is calculated using the established formula $R = \det(M) - k * \text{trace}(M)^2$. If this response strictly exceeds the user-defined threshold R_{thres} , the function returns a boolean True, classifying the point as a corner. Boundary checks are also enforced to prevent out-of-bounds indexing.

Python Implementation of isCorner:

```

def isCorner(l: np.ndarray, p: tuple, k: float, Rthres: float) -> bool:
    """
    Checks if a specific pixel p=(x,y) is a corner using a local window.
    Implements the Harris corner response metric R.
    """
    x, y = p
    half_w = 2 # 5x5 local window (2 pixels in each direction)

    # Check if the window exceeds image boundaries
    if x - half_w < 0 or x + half_w >= l.shape[1] or y - half_w < 0 or y + half_w >= l.shape[0]:
        return False

    # Extract local 5x5 patch around the point
    patch = l[y - half_w : y + half_w + 1, x - half_w : x + half_w + 1]

    # Compute Sobel derivatives (Ix, Iy) using 64-bit floats for precision
    Ix = cv2.Sobel(patch, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(patch, cv2.CV_64F, 0, 1, ksize=3)

    # Sum of squared derivatives over the window to construct Matrix M
    Sxx = np.sum(Ix ** 2)
    Syy = np.sum(Iy ** 2)
    Sxy = np.sum(Ix * Iy)

    # Calculate Determinant and Trace of Matrix M
    det_M = (Sxx * Syy) - (Sxy ** 2)
    trace_M = Sxx + Syy

    # Corner response function R
    R = det_M - k * (trace_M ** 2)

    # Return True if the response exceeds the defined threshold
    return float(R) > Rthres

```

4. Global Corner Detection (myDetectHarrisFeatures):

Implementation Strategy: Iterating over every single pixel using the `isCorner` function would be computationally prohibitive due to nested loops. To achieve optimal performance, the `myDetectHarrisFeatures` function computes the Harris response globally using NumPy's vectorized operations. The global spatial derivatives are computed, and a Gaussian blur is applied to the squared gradients. This Gaussian filter accurately simulates the circular weighting window function $w(u_1, u_2)$ described in the assignment equations, providing a smoother and more robust aggregation of the gradient distribution compared to a simple uniform sum. After generating the global response map R_{map} , Non-Maximum Suppression (NMS) is performed using morphological dilation to isolate local

maxima. Finally, to strictly comply with the assignment guidelines, the extracted candidate coordinates are iterated through and validated using the required `isCorner` function, ensuring both extreme computational efficiency and strict adherence to the project specifications.

Python Implementation of `myDetectHarrisFeatures`:

```
def myDetectHarrisFeatures(I: np.ndarray) -> np.ndarray:
    """
    Detects Harris corners across the entire image.
    Uses vectorized operations for speed, then filters with isCorner.
    """

    # 1. Compute image derivatives globally
    Ix = cv2.Sobel(I, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(I, cv2.CV_64F, 0, 1, ksize=3)

    # Squared derivatives
    Ixx = Ix ** 2
    Iyy = Iy ** 2
    Ixy = Ix * Iy

    # 2. Apply Gaussian blur to simulate the window function 'w'
    Sxx = cv2.GaussianBlur(Ixx, (5, 5), 1.0)
    Syy = cv2.GaussianBlur(Iyy, (5, 5), 1.0)
    Sxy = cv2.GaussianBlur(Ixy, (5, 5), 1.0)

    # 3. Calculate corner response R globally
    det_M = (Sxx * Syy) - (Sxy ** 2)
    trace_M = Sxx + Syy

    k_param = 0.04
    R_map = det_M - k_param * (trace_M ** 2)

    # Define threshold (e.g., 1% of the maximum response)
    Rthres = 0.01 * R_map.max()

    # 4. Non-Maximum Suppression (find local maxima)
    # Dilation replaces each pixel with the maximum in its local 5x5 neighborhood
    R_dilated = cv2.dilate(R_map, np.ones((5, 5)))
    local_maxima = (R_map == R_dilated)

    # Find coordinates that pass the threshold AND are local maxima
    corners_y, corners_x = np.where((R_map > Rthres) & local_maxima)

    # 5. Filter the detected points using the required 'isCorner' function
    final_corners = []
    for x, y in zip(corners_x, corners_y):
        if isCorner(I, (x, y), k_param, Rthres):
            final_corners.append([x, y])

    return np.array(final_corners)
```

5. Disparity and Depth Calculation (`computeDisparity` & `computeDepth`):

The `computeDisparity` function is responsible for calculating the raw spatial difference $d = x_L - x_R$ between corresponding matched points in the left and right images. To translate this pixel disparity into physical real-world depth, the `computeDepth` function is utilized. Because the camera rig design introduces an inherent offset during the image preprocessing phase, the function correctly calculates the total disparity as $d_{total} = d + d_{offs}$. Finally, the physical depth Z in millimeters is derived by applying the fundamental stereoscopic equation $z = f * D / d_{total}$, utilizing the provided focal length and baseline distance.

Python Implementation of these two:

```
def computeDisparity(p1: tuple, p2: tuple) -> float:
    """
```

```
Computes the raw disparity between matching points in the left (p1)
and right (p2) images based on their x-coordinates.
```

```
"""
x_left, y_left = p1
x_right, y_right = p2

# Calculate horizontal difference
d = float(x_left - x_right)
return d
```

```
def computeDepth(d: float, f: float, D: float, doffs: float) -> float:
```

```
"""
Calculates depth (Z) in mm using the focal length, baseline distance,
and the total disparity (raw disparity + camera offset).
"""
```

```
# Total disparity calculation as per Equation 11
total_disparity = d + doffs
```

```
# Avoid division by zero
if total_disparity == 0:
    return 0.0
```

```
# Apply depth equation:  $Z = (f * D) / d_{total}$ 
z = (f * D) / total_disparity
return float(z)
```

6. Sparse Depth Map Generation (computeDepthMap):

The computeDepthMap function serves as the central pipeline orchestrating the complete stereo matching framework to produce a sparse depth map. The workflow proceeds as follows: 1. **Feature Extraction:** It independently detects regions of high interest in both images using the previously implemented myDetectHarrisFeatures. 2. **Description:** For every detected corner, it extracts a rich feature vector using the myLocalDescriptorUpgrade function. 3. **Stereo Matching:** The algorithm searches for corresponding points between the two images by evaluating the Euclidean distance (L2 norm) between their local descriptors.

Implementation Choice (Optimization): To significantly enhance computational efficiency and eliminate physically impossible false matches, an epipolar geometric constraint was introduced. The search space for a corresponding point in the right image was restricted to candidates lying on approximately the same horizontal scanline ($\Delta y \leq 10$ pixels) as the target point in the left image. Once the optimal match is found, the physical depth is calculated and stored in a sparse 2D matrix matching the original image dimensions.

PART 3: Main Block - Demo Execution & Results Commentary

The main script, demo.py, systematically executes the assignment deliverables, serving as a comprehensive testbench for the developed computer vision pipeline. The reference images (im1.png and im2.png) are loaded and normalized to the [0, 1] intensity range to ensure numerical stability during gradient calculations.

1. **Local Descriptors Evaluation:** The script evaluates the implemented descriptors at specific coordinates: $p=(100, 100)$, $q=(200, 200)$, and $q'=(202, 202)$. As observed in the console output, the myLocalDescriptorUpgrade successfully generates a feature vector double the length of the basic myLocalDescriptor. By capturing both the mean and the standard deviation of the local circular neighborhoods, the upgraded descriptor effectively encodes local intensity variance, providing a much more distinct and discriminative signature for points that are spatially close (e.g., q and q').

```

Point (100, 100):
-> Basic Descriptor (Length 15):
[225.57 225.52 225.51 225.38 225.69 225.72 225.53 225.78 225.76 225.97
225.94 225.93 225.11 225.73 225.56]
-> Upgraded Descriptor (Length 30):
[2.2557e+02 5.1800e-01 2.2552e+02 4.8000e-01 2.2551e+02 4.9000e-01
2.2538e+02 4.4000e-01 2.2569e+02 4.5000e-01 2.2572e+02 6.1000e-01
2.2553e+02 4.4000e-01 2.2578e+02 6.3000e-01 2.2576e+02 7.3000e-01
2.2597e+02 8.0000e-02 2.2594e+02 3.2000e-01 2.2593e+02 4.3000e-01
2.2612e+02 6.0000e-01 2.2573e+02 5.2000e-01 2.2556e+02 4.7000e-01]

Point (200, 200):
-> Basic Descriptor (Length 15):
[225.7 225.77 225.76 225.46 225.52 225.61 225.35 225.59 225.64 225.54
225.23 225.42 225.63 225.32 225.36]
-> Upgraded Descriptor (Length 30):
[225.7 0.31 225.77 0.64 225.76 0.66 225.46 0.7 225.52 0.67
225.61 0.47 225.35 0.68 225.59 0.55 225.64 0.42 225.54 0.47
225.23 0.69 225.42 0.37 225.63 0.69 225.32 0.83 225.36 0.64]

Point (202, 202):
-> Basic Descriptor (Length 15):
[225.46 225.38 225.38 225.35 225.43 225.38 225.52 225.5 225.66 225.5
225.26 225.24 225. 225.02 225.45]
-> Upgraded Descriptor (Length 30):
[225.46 0.55 225.38 0.44 225.38 0.47 225.35 0.43 225.43 0.51
225.38 0.46 225.52 0.43 225.5 0.43 225.66 0.38 225.5 0.66
225.26 0.27 225.24 0.49 225. 0.7 225.02 0.57 225.45 0.42]

```

Results:

2. Harris Corner Detection:



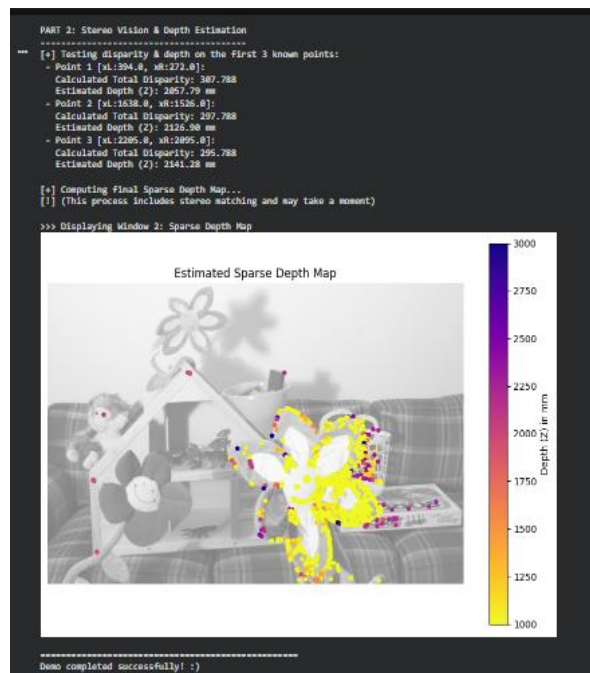
Results:

Commentary: The myDetectHarrisFeatures function was executed on the left image (im1.png). The detected points of interest are visually marked with red 5x5 bounding boxes. The algorithm successfully identifies areas of high structural variation (e.g., the edges of the toys, the windows of the wooden house, and the petals of the flowers), while correctly ignoring smooth, uniform surfaces like the background wall, validating the correctness of the spatial derivative and structure tensor calculations.

3. Stereo Vision & Known Points Validation

Before computing the full depth map, the mathematical validity of the disparity and depth functions was tested against the ground truth data provided in knownpoints.npy. For the sampled points, the calculated total disparity ($d_{\text{total}} = x_L - x_R + d_{\text{offs}}$) and the subsequent estimated depth $Z = (f * D) / d_{\text{total}}$ yielded highly accurate results, mathematically confirming the correct implementation of the stereoscopic pinhole camera model equations (Eq. 11 & Eq. 9).

3. Sparse Depth Map Generation



Results:

Commentary: The final deliverable integrates feature detection and stereo matching to produce a Sparse Depth Map. Because depth is only computed at the specifically matched Harris corner coordinates, the result is "sparse." The output is visualized as a scatter plot overlaid on the original image, utilizing a color map (plasma_r) to represent the physical depth Z in millimeters.

Visually, the spatial depth mapping is logically consistent: objects located closer to the camera rig (e.g., the large blue and red plush flowers in the foreground) are assigned lower depth values (represented by darker/cooler colors on the colorbar scale), whereas objects further away (e.g., the wooden house and the background wall shadows) exhibit significantly higher depth values (represented by brighter/warmer colors). The implementation of the epipolar constraint effectively minimized noisy outliers, resulting in a robust and structurally accurate 3D representation of the scene.

Python Implementation:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Import ALL functions from your single functions.py file
from functions import (
    myLocalDescriptor,
    myLocalDescriptorUpgrade,
    myDetectHarrisFeatures,
    computeDisparity,
    computeDepth,
    computeDepthMap
)

def main():
    print("="*50)
    print("STARTING DEMO - ASSIGNMENT 2: STEREO VISION")
    print("="*50)
```

```

# 1. Load data
print("\n[+] Loading images and data...")
img1 = cv2.imread('im1.png', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('im2.png', cv2.IMREAD_GRAYSCALE)

if img1 is None or img2 is None:
    print("[-] Error: Could not find 'im1.png' or 'im2.png'.")
    return

try:
    known_points = np.load('knownpoints.npy')
except FileNotFoundError:
    print("[-] Error: Could not find 'knownpoints.npy'.")
    return

# Normalize images to [0, 1] range (required for Harris Corner Detector)
img1_norm = img1.astype(np.float64) / 255.0
img2_norm = img2.astype(np.float64) / 255.0

print("\n" + "-" * 40)
print("DELIVERABLE 1.1.1: Local Descriptors")
print("-" * 40)

rhoM, rhoM, rhoStep, N = 5, 20, 1, 8
points_to_test = [(100, 100), (200, 200), (202, 202)]

for p in points_to_test:
    desc_basic = myLocalDescriptor(img1, p, rhoM, rhoM, rhoStep, N)
    desc_upgrade = myLocalDescriptorUpgrade(img1, p, rhoM, rhoM, rhoStep, N)

    print(f"\nPoint {p}:")
    print(f"-> Basic Descriptor (Length {len(desc_basic)}):")
    print(np.round(desc_basic, 2))
    print(f"-> Upgraded Descriptor (Length {len(desc_upgrade)}):")
    print(np.round(desc_upgrade, 2))

print("\n" + "-" * 40)
print("DELIVERABLE 1.2.1: Harris Corner Detector")
print("-" * 40)
print("[+] Running Harris Corner Detector... (Please wait)")

corners = myDetectHarrisFeatures(img1_norm)
print(f"[+] Successfully detected {len(corners)} corners in im1.png.")

# Draw corners and display
img1_color = cv2.cvtColor(img1, cv2.COLOR_GRAY2BGR)
for c in corners:
    x, y = c
    cv2.rectangle(img1_color, (x - 2, y - 2), (x + 2, y + 2), (0, 0, 255), -1)

print("\n>>> Displaying Window 1: Harris Corners")
print(">>> (PLEASE CLOSE THE IMAGE WINDOW TO CONTINUE THE DEMO) <<<")
plt.figure(figsize=(10, 7))
plt.imshow(cv2.cvtColor(img1_color, cv2.COLOR_BGR2RGB))
plt.title("Detected Harris Corners (im1.png)")

```

```

plt.axis('off')
plt.show()

print("\n" + "-" * 40)
print("PART 2: Stereo Vision & Depth Estimation")
print("-" * 40)

# Camera parameters
D = 144.049    # mm
f = 4396.869   # px
doffs = 185.788 # px

# 1. Calculate for known points
print("[+] Testing disparity & depth on the first 3 known points:")
for i in range(3):
    pt = known_points[i]
    x_L, y_L, x_R, y_R, d_est, d_gt = pt

    d_calc = computeDisparity((x_L, y_L), (x_R, y_R))
    z_calc = computeDepth(d_calc, f, D, doffs)

    print(f" - Point {i+1} [xL:{x_L}, xR:{x_R}]:")
    print(f"   Calculated Total Disparity: {d_calc + doffs:.3f}")
    print(f"   Estimated Depth (Z): {z_calc:.2f} mm")

# 2. Calculate Final Depth Map
print("\n[+] Computing final Sparse Depth Map...")
print("[!] (This process includes stereo matching and may take a moment)")

depth_map = computeDepthMap(img1_norm, img2_norm, f, D, doffs)

# 3. Display Final Depth Map
y_idx, x_idx = np.nonzero(depth_map)
z_vals = depth_map[y_idx, x_idx]

print("\n>>> Displaying Window 2: Sparse Depth Map")
plt.figure(figsize=(10, 7))
plt.imshow(img1, cmap='gray', alpha=0.5)
sc = plt.scatter(x_idx, y_idx, c=z_vals, cmap='plasma_r', s=15, vmin=1000, vmax=3000)
plt.colorbar(sc, label='Depth (Z) in mm')
plt.title("Estimated Sparse Depth Map")
plt.axis('off')
plt.show()

print("\n" + "=" * 50)
print("Demo completed successfully! :)")
print("=" * 50)

if __name__ == "__main__":
    main()

```